

***BRAC*'s**

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 7

Roll no: 66 PRN No: 12411609 Name of Student: Laukika Shinde

Subject Name: Deep Learning

Problem Statement: Implement transfer learning using pre-trained AlexNet, VGG16, ResNet50, and EfficientNetB0 models for image classification, and compare their performance.

Algorithm:

- 1) Import the required libraries and pretrained models.
- 2) Load the CIFAR-10 image classification dataset.
- 3) Resize the images to 224×224 pixels and normalize them using ImageNet statistics.
- 4) Load pretrained AlexNet, VGG16, ResNet50 and EfficientNetB0 models.
- 5) Freeze the pretrained layers of each model.
- 6) Replace the final classification layer with a 10-class output layer.
- 7) Compile the models using cross-entropy loss and the Adam optimizer.
- 8) Train each model on the training dataset.
- 9) Evaluate each model on the test dataset.
- 10) Record the accuracy and loss of each model.
- 11) Compare the performance of all four models using a bar graph.

Code:

```
import torch
```

```
import torchvision
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from torch import nn, optim
```

```
from torch.utils.data import DataLoader, Subset
```

```
from torchvision import datasets, transforms, models
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Device:", device)
```

```
transform = transforms.Compose([
```

```
    transforms.Resize((224, 224)),
```

```
    transforms.ToTensor(),
```

```
    transforms.Normalize(
```

```
        mean=[0.485, 0.456, 0.406],
```

```
        std=[0.229, 0.224, 0.225]
```

```
    )
```

```
])
```

```
train_dataset = datasets.CIFAR10(
```

```
    root="./data",
```

```
    train=True,
```

```
    download=True,
```

```
    transform=transform
```

```
)
```

```
test_dataset = datasets.CIFAR10(
```

```
    root="./data",
```

```
    train=False,
```

```
download=True,  
transform=transform  
)
```

```
train_dataset = Subset(train_dataset, range(10000))  
test_dataset = Subset(test_dataset, range(2000))
```

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=64,  
    shuffle=True,  
    num_workers=2  
)
```

```
test_loader = DataLoader(  
    test_dataset,  
    batch_size=64,  
    shuffle=False,  
    num_workers=2  
)
```

```
alexnet = models.alexnet(weights=models.AlexNet_Weights.DEFAULT)  
vgg16 = models.vgg16(weights=models.VGG16_Weights.DEFAULT)  
resnet50 = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
```

```
efficientnet = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
```

```
def prepare_model(model, name):
```

```
    for param in model.parameters():
```

```
        param.requires_grad = False
```

```
    if name == "AlexNet":
```

```
        model.classifier[6] = nn.Linear(model.classifier[6].in_features, 10)
```

```
        trainable = model.classifier[6].parameters()
```

```
    elif name == "VGG16":
```

```
        model.classifier[6] = nn.Linear(model.classifier[6].in_features, 10)
```

```
        trainable = model.classifier[6].parameters()
```

```
    elif name == "ResNet50":
```

```
        model.fc = nn.Linear(model.fc.in_features, 10)
```

```
        trainable = model.fc.parameters()
```

```
    else:
```

```
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, 10)
```

```
        trainable = model.classifier[1].parameters()
```

```
    model = model.to(device)
```

```
    return model, trainable
```

```
models_dict = {}
```

```
alexnet, alexnet_trainable = prepare_model(alexnet, "AlexNet")
```

```
vgg16, vgg16_trainable = prepare_model(vgg16, "VGG16")
```

```
resnet50, resnet50_trainable = prepare_model(resnet50, "ResNet50")
```

```
efficientnet, efficientnet_trainable = prepare_model(efficientnet, "EfficientNetB0")
```

```
models_dict["AlexNet"] = (alexnet, alexnet_trainable)
```

```
models_dict["VGG16"] = (vgg16, vgg16_trainable)
```

```
models_dict["ResNet50"] = (resnet50, resnet50_trainable)
```

```
models_dict["EfficientNetB0"] = (efficientnet, efficientnet_trainable)
```

```
criterion = nn.CrossEntropyLoss()
```

```
results = []
```

```
for name, (model, trainable) in models_dict.items():
```

```
    optimizer = optim.Adam(trainable, lr=0.001)
```

```
    for epoch in range(3):
```

```
        model.train()
```

```
        running_loss = 0
```

```
for images, labels in train_loader:
```

```
    images, labels = images.to(device), labels.to(device)
```

```
    optimizer.zero_grad()
```

```
    outputs = model(images)
```

```
    loss = criterion(outputs, labels)
```

```
    loss.backward()
```

```
    optimizer.step()
```

```
    running_loss += loss.item()
```

```
print(name, "Epoch", epoch + 1, "Loss:", running_loss / len(train_loader))
```

```
model.eval()
```

```
correct = 0
```

```
total = 0
```

```
test_loss = 0
```

```
with torch.no_grad():
```

```
    for images, labels in test_loader:
```

```
        images, labels = images.to(device), labels.to(device)
```

```
        outputs = model(images)
```

```
        loss = criterion(outputs, labels)
```

```
test_loss += loss.item()

_, predicted = torch.max(outputs, 1)

total += labels.size(0)

correct += (predicted == labels).sum().item()


accuracy = correct / total

loss_value = test_loss / len(test_loader)

results.append([name, accuracy, loss_value])

print(name, "Test Accuracy:", accuracy)

print(name, "Test Loss:", loss_value)


results_df = pd.DataFrame(

    results,

    columns=["Model", "Test Accuracy", "Test Loss"]

)

print(results_df)


plt.figure(figsize=(10, 6))

plt.bar(results_df["Model"], results_df["Test Accuracy"])
```

```
plt.xlabel("Model")

plt.ylabel("Test Accuracy")

plt.title("Comparison of Transfer Learning Models")

plt.ylim(0, 1)

plt.xticks(rotation=15)

plt.show()
```

Output:

```
Device: cuda
100%|██████████| 170M/170M [00:11<00:00, 14.6MB/s]
Downloading: "https://download.pytorch.org/models/alexnet-owt-7be5be79.pth" to /root/.cache/torch/hub/checkpoints/alexnet-owt-7be5be79.pth
100%|██████████| 233M/233M [00:02<00:00, 105MB/s]
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth
100%|██████████| 528M/528M [00:05<00:00, 98.1MB/s]
Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 154MB/s]
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwightman-7f5810bc.pth
100%|██████████| 20.5M/20.5M [00:00<00:00, 150MB/s]
```

```
AlexNet Epoch 1 Loss: 0.8599930439785028
AlexNet Epoch 2 Loss: 0.6302723339788473
AlexNet Epoch 3 Loss: 0.5580676120177955
AlexNet Test Accuracy: 0.782
AlexNet Test Loss: 0.6038732025772333
VGG16 Epoch 1 Loss: 0.7809735775753192
VGG16 Epoch 2 Loss: 0.5808885862493212
VGG16 Epoch 3 Loss: 0.5405271116905151
VGG16 Test Accuracy: 0.8115
VGG16 Test Loss: 0.5112502658739686
ResNet50 Epoch 1 Loss: 1.2717530529969816
ResNet50 Epoch 2 Loss: 0.7904477313065984
ResNet50 Epoch 3 Loss: 0.678027433764403
ResNet50 Test Accuracy: 0.7755
ResNet50 Test Loss: 0.6836346108466387
EfficientNetB0 Epoch 1 Loss: 1.3424380827861226
EfficientNetB0 Epoch 2 Loss: 0.8518326608998001
EfficientNetB0 Epoch 3 Loss: 0.7610091445552316
EfficientNetB0 Test Accuracy: 0.7745
EfficientNetB0 Test Loss: 0.6836326774209738
```

	Model	Test Accuracy	Test Loss
0	AlexNet	0.7820	0.603873
1	VGG16	0.8115	0.511250
2	ResNet50	0.7755	0.683635
3	EfficientNetB0	0.7745	0.683633

